

Application web Interactive avec Logique Métier (Jeux de prédictions sportifs)

« Top 5 League »

Loubatiere Alexis

Formation : Développeur Web et Web Mobile Full-stack

Centre de formation : Centre Européen de Formation (CEF)

Sommaire

	<u>n° de page</u>
<u>Compétences couvertes par le projet</u>	4
<u>Contexte du projet :</u>	
- Présentation d'une entreprise potentielle	5
- Présentation du projet	6
- Cahier des charges	7
- Contraintes du projet :	
Responsive Design	8
Accessibilité	9
Sécurité	9
Protection des données	10
- Livrable attendus	11
- Environnement humain et techniques	11
<u>Développement de la partie Front-End :</u>	
- Environnement choisis	12
- Sécurité	12
- Maquettes :	
Exemple de maquettes desktop et mobile	13 - 16
Schéma de l'enchaînement des maquettes	17 - 18
- Référencement SEO	19
- Organisations des composants	20
- Captures d'écran d'interfaces utilisateurs (web et web-mobile)	21 - 23
- Exemple d'interface statique	24 - 25
- Exemple d'interface dynamique	26 - 27
<u>Développement de la partie Back-End :</u>	
-Base de données / MCD / MLD :	
Environnement choisis pour la Base de données	28
MCD	28 - 29
MLD	29

Script de création SQL	30
- Environnement choisis	31
- Éléments de sécurité	32
- Extraits de code de composant métier	33
- Extrait de code de composant d'accès aux données	34
- Présentation du jeu d'essai de la fonctionnalité la plus représentative	35 - 38
- Déploiement de l'application	39
- Tests supplémentaire réalisés (tests unitaires)	39
- Veille sur les vulnérabilités de sécurité	40
 <u>Conclusion :</u>	
- Difficultés rencontrées	41
- Amélioration possibles	41
- Remerciement	42
 <u>Annexes :</u>	
- Maquettes des interfaces utilisateur	43 - 44
- Captures d'écran d'une interface utilisateur et le code correspondant	45 - 46
- Code de composants métier les plus significatifs	47 - 48
- Code de composants d'accès aux données les plus significatifs	49

Compétences couvertes par le projet

Ce projet a permis de développer et consolider des compétences complètes en développement web full-stack, aussi bien côté back-end que front-end.

De manière générale, il a permis de gérer seul un projet full-stack en organisant le code et le développement et en créant une logique métier complexe. Il m'a également permis l'utilisation d'outil de gestion de version (Git et GitHub) indispensable pour l'organisation propre du projet. La ou il m'a le plus apporté c'est au niveau du débogage, de la gestion de problèmes ainsi que le travail autour des test et de la maintenabilité de l'API.

Pour ce qui est du back-end, ce projet a permis de concevoir et développer une API REST complète avec Node.js et Express.js. Il a également permis la création et la gestion d'une base de données SQL ainsi que le développement des fonctionnalités permettant de créer, consulter, modifier et supprimer les données (opérations CRUD). Un système d'authentification sécurisé utilisant JWT a été mis en place, accompagné d'une gestion des rôles et autorisations pour différencier les visiteurs, utilisateurs et administrateurs.

Une application structurée selon une architecture MVC claire et maintenable a pu être réalisée tout en mettant en place des services dédiés à la gestion des différentes règles métier et fonctionnalités de l'application. Des validations de données et une gestion des erreurs ont été mises en place afin d'assurer la fiabilité de l'application. Des tests unitaires ont également été réalisés avec Jest et Supertest afin de vérifier le bon fonctionnement des différentes fonctionnalités de l'API.

Enfin, ce projet a permis de gérer les échanges avec la base de données de manière sécurisée et cohérente, tout en mettant en place les relations (plus ou moins complexes) entre les différents modèles de l'application. Une documentation via le fichier README a également été mise en place afin de faciliter la compréhension et l'utilisation du projet.

Pour ce qui est du front-end, ce projet a permis de développer une interface dynamique avec React. J'ai travaillé sur la création de composants réutilisables, la gestion de la navigation avec React Router DOM ainsi que l'affichage dynamique des données provenant de l'API.

Le projet a également permis de gérer les formulaires utilisateurs (utilisation fetch et axios pour l'envoi de donnée), l'authentification côté client et l'organisation des différentes pages de l'application afin de rendre la navigation plus fluide. Enfin, j'ai utilisé HTML, CSS et JavaScript pour créer une interface responsive, claire et agréable à utiliser.

Contexte du projet

Présentation d'une potentielle entreprise pour ce projet :

Dans le cadre de ce projet réaliser seul et non dans le cas d'un stage, l'entreprise fictive imaginée pourrait être une plateforme spécialisée dans les challenges sportifs autour de la NBA. Son objectif serait de proposer aux utilisateurs une expérience interactive leur permettant de créer chaque mois leur propre "Top 5" de joueurs en fonction de leurs performances statistiques.

L'entreprise aurait pour but de réunir une communauté de passionnés de basketball autour d'un système de prédictions et de classements mensuels. Les utilisateurs pourraient comparer leurs choix avec ceux des autres participants ainsi qu'avec le Top 5 officiel publié à la fin de chaque challenge.

Cette plateforme proposerait également un espace administrateur permettant de gérer les joueurs du mois, les différents challenges mensuels ainsi que les résultats officiels. L'objectif principal serait d'offrir une application moderne, dynamique et simple d'utilisation pour les fans de NBA souhaitant tester leurs connaissances et suivre les performances des joueurs tout au long de la saison.

Présentation du projet :

J'ai décidé ici de lier mes 2 passions : l'informatique et le basketball. Quoi de mieux pour cela que de créer un site dédié à un concept de fantasy basketball, permettant de combiner analyse de données et passion du jeu.

Le projet consiste donc à concevoir et développer une application web de type jeu de prédiction sportive « basket fantasy », centrée sur les performances réelles de joueurs de basketball de manière mensuelle.

L'objectif principal est de proposer une plateforme interactive permettant aux utilisateurs de constituer, chaque mois, un "Top 5" de joueurs qu'ils pourront sélectionner parmi 20 joueurs postés chaque début de mois en estimant leurs statistiques (points, passes décisives et rebonds), puis de comparer leurs prédictions aux performances réelles afin d'établir un classement et de désigner un gagnant.

L'application intègre un système d'authentification et de gestion des rôles, distinguant les visiteurs, les utilisateurs et les administrateurs. Les visiteurs auront accès aux résultats du mois précédent, aux règles du jeu, aux informations du challenge en cours et aux joueurs possible de sélectionner pour le mois en cours. Les utilisateurs, en plus d'avoir accès à ce que les visiteurs auront, pourront créer et gérer leurs Top 5. Enfin, les administrateur disposeront d'un espace dédié leur permettant de gérer les challenges mensuels, de consulter la liste des utilisateurs, d'alimenter la base de joueurs et de publier les résultats officiels.

Ce projet s'inscrit dans le cadre de la formation Développeur Web et Web Mobile et vise à mobiliser des compétences fullstack, notamment en développement front-end et back-end, en conception et gestion de bases de données relationnelles, ainsi qu'en sécurisation et structuration d'une application web.

Le projet repose donc sur une logique de comparaison entre données prédites et données réelles.

Cahier des Charges :

Le projet consiste à développer une application web permettant aux utilisateurs de participer à un jeu de prédiction basé sur les performances de joueurs de basketball. L'objectif est de proposer une API interactive où chaque utilisateur connecté peut constituer un "Top 5" mensuel de joueurs choisis parmi 20 joueurs proposé en estimant leurs statistiques (points, passes décisives, rebonds), puis comparer ses prédictions aux résultats réels afin de déterminer un classement et un gagnant. Cette application est donc faite en premier pour les fans de basketball mais également pour les fans de sports et de statistiques sportives tout simplement.

Chaque type d'utilisateurs aura différentes fonctionnalités disponibles :

Les visiteurs :

- Inscription / Connexion
- Accès aux résultats et au gagnant du mois précédent
- Accès aux joueurs du mois
- Accès aux règles du jeu
- Accès aux informations du challenge en cours

Les utilisateurs connectés (en plus des fonctionnalités disponible pour les visiteurs) :

- Création, modification et consultation de son Top 5
- Si participation lors du challenge du mois précédent, possibilité de comparer son Top 5 avec le Top 5 officiel et le Top 5 gagnant.

Les administrateurs :

- Création et gestion des challenges mensuels
- Accès à la liste des utilisateurs
- Publication du Top 5 officiel (résultats réels)
- Détermination et validation du gagnant
- Accès à un tableau de gestion des challenges (informations challenge en cours, historique des challenges)

Contraintes du projet :

Responsive Design

Le responsive design a été mis en place afin de garantir une expérience utilisateur (UX) optimale, quel que soit le support utilisé. L'interface s'adapte automatiquement à la taille de l'écran grâce à l'utilisation de grilles flexibles et de composants réactifs.

Pour cela, plusieurs solutions ont été utilisées :

- Utilisation du système de grille Bootstrap (rows / columns) permettant d'organiser les éléments
- Mise en place de classes responsives (col-md, col-lg, etc.) pour adapter l'affichage selon la taille de l'écran
- Utilisation de Flexbox pour gérer l'alignement et la disposition des composants
- Adaptation des marges, espacements et tailles de texte pour conserver une bonne lisibilité
- Utilisation d'un logo différent et d'un menu burger selon la taille de l'écran

Si on prend en exemple la page d'affichage des joueurs du mois, les joueurs sont affichés par poste sur une ligne (4 joueurs sur une ligne), tandis qu'ils se réorganisent en colonne sur mobile afin de les garder dans des proportions leur permettant d'être correctement lisible.

Ces choix permettent d'assurer une interface cohérente, accessible et « user friendly » sur l'ensemble du site.

Accessibilité

Afin de rendre l'interface accessible au plus grand nombre, plusieurs bonnes pratiques ont été mises en place :

- utilisation de balises HTML sémantiques (titres, formulaires, labels) pour structurer le contenu
- association des champs de formulaire avec des labels et des placeholders afin de faciliter la compréhension et l'utilisation
- mise en place d'une hiérarchie claire des titres pour améliorer la lisibilité
- choix d'une palette de couleurs réfléchie afin de guider l'utilisateur et de mettre en évidence les éléments importants de l'interface
- utilisation de l'attribut « alt » sur les images afin de fournir une alternative textuelle si celles-ci ne chargent pas
- conception d'une interface simple et intuitive permettant une navigation fluide

Par exemple, pour tous les formulaires, chaque champ est clairement identifié et accompagné d'un label, facilitant la saisie des informations par l'utilisateur.

Ces choix permettent d'améliorer l'accessibilité globale de l'application et de proposer une expérience utilisateur (UX) plus large.

Sécurité

Plusieurs éléments liés à la sécurité ont été mis en place dans l'application afin de protéger les données et les accès utilisateurs. L'authentification est gérée grâce aux tokens JWT, tandis que les mots de passe sont sécurisés avec bcrypt avant leur stockage en base de données. Des rôles et autorisations permettent également de limiter certaines fonctionnalités aux administrateurs.

La validation des données utilisateurs a été réalisée avec express-validator afin de contrôler les informations envoyées à l'API et limiter les erreurs ou données invalides. Des protections supplémentaires ont également été ajoutées avec Helmet pour renforcer la sécurité des échanges HTTP ainsi qu'avec CORS pour contrôler les communications entre le front-end React et l'API back-end.

Enfin, une attention particulière a été portée à la cohérence et à la sécurité des données grâce à l'utilisation de transactions SQL, de contrôles métier et d'une gestion adaptée des erreurs côté serveur.

Protection des données

Pour ce qui est de la protection des données, plusieurs mesures ont été mises en place afin de la garantir. Tout d'abord, un système d'authentification et de gestion des rôles permet de restreindre l'accès aux différentes parties de l'application. Chaque utilisateur ne peut accéder qu'aux fonctionnalités qui lui sont autorisées (visiteur, utilisateur ou administrateur). De plus, les utilisateurs connectés ne peuvent consulter et manipuler que leurs propres données, ce qui garantit une isolation des informations entre les comptes.

Par ailleurs, des validations sont effectuées côté serveur afin de contrôler les données envoyées à l'API et d'éviter l'insertion de données incorrectes. Ces validations sont réalisées à la fois de manière manuelle dans les services et à l'aide d'un système de validation dédié (validator), permettant de vérifier le format, la présence et la cohérence des données avant leur traitement.

Ces différentes mesures contribuent à assurer la confidentialité, l'intégrité et la sécurité des données manipulées au sein de l'application.

Livrable attendu

Les livrables attendus dans le cadre du projet sont les suivants :

- une application front-end fonctionnelle ;
- une API back-end sécurisée ;
- une base de données MySQL structurée ;
- un dépôt GitHub contenant le code source ;
- un dossier projet détaillant la démarche de réalisation ;
- un support de présentation pour la soutenance.

Ces livrables permettent de démontrer la maîtrise des différentes compétences visées par la formation.

Environnement Technique et humain

Le projet a été réalisé individuellement dans le cadre de la formation Développeur Web et Web Mobile full-stack.

L'environnement technique utilisé est le suivant :

- Visual Studio Code pour le développement ;
- Node.js et npm pour la gestion des dépendances ;
- React pour l'interface utilisateur ;
- Express pour le serveur API ;
- MySQL et phpMyAdmin pour la base de données ;
- Git et GitHub pour le versionnement du code ;
- Figma pour les maquettes ;
- Libre Office pour la rédaction du dossier projet et la confection du Powerpoint.

Pour les tests :

- Jest et Supertest pour les test « automatiques » (contrôle des routes API, des réponses du serveur ainsi que des différentes fonctionnalités métiers) ;
- Postman pour les test « manuel » (pour effectuer des tests manuels des endpoints, vérifier les requêtes HTTP et contrôler les échanges entre le front-end et l'API) ;
- Nodemon a été utilisé afin de relancer automatiquement le serveur lors de modification du code.

Développement de la partie Front-end

Environnement choisis

Pour le développement du front-end, l'environnement React a été choisi afin de créer une interface moderne, dynamique et réactive. React a permis de structurer l'application en composants réutilisables, ce qui a facilité l'organisation du projet et la maintenance du code. La navigation entre les différentes pages a été mise en place grâce à React Router DOM afin d'offrir une expérience utilisateur plus fluide. Des outils comme Axios ont également été utilisés pour communiquer avec l'API back-end et récupérer les différentes données de l'application. Enfin, Bootstrap, Bootstrap Icons et Sass ont permis de concevoir une interface responsive, claire et adaptée à différents types d'écrans.

Sécurité

Concernant la sécurité côté front-end, plusieurs mécanismes ont été mis en place afin de protéger l'accès à l'application et les données utilisateurs. La gestion de l'authentification a été réalisée avec les tokens JWT afin de sécuriser les sessions utilisateurs et limiter l'accès aux pages protégées. Des contrôles ont également été ajoutés côté client pour gérer les rôles et empêcher l'accès à certaines fonctionnalités aux utilisateurs non autorisés.

Une attention particulière a aussi été portée à la validation des formulaires afin de limiter les erreurs de saisie avant l'envoi des données à l'API. Enfin, la communication entre le front-end React et le back-end a été sécurisée grâce à l'utilisation des tokens JWT et à la mise en place de contrôles d'accès selon les droits des utilisateurs.

Maquettes :

Avant le développement de l'application, des maquettes ont été réalisées sur Figma afin de concevoir l'interface utilisateur et l'expérience globale de l'application.

Etant donné le nombre de maquettes réalisé (différentes pour chaque rôle) je vous présenterai ici uniquement les maquettes pour le rôle « visiteur » en version desktop et mobile. Néanmoins, celles ci permettent bien d'illustrer la réflexion menée autour de l'organisation des interfaces pour l'ensemble du projet et je vous mettrai en annexe le lien menant à la page Figma sur laquelle figure toutes les maquettes.

Exemple de maquettes desktop et mobile

Mobile :

Page inscription

The registration screen features a dark blue header with the 'TOP 5 LEAGUE' logo on the left and a hamburger menu icon on the right. The main content area is light yellow and contains a white registration form with a blue border. The form has the title 'Inscription' in bold blue text. It includes four input fields labeled 'Pseudo :', 'Joueur préféré :', 'Email :', and 'Mot de passe :'. Below these fields is a blue button labeled 'S'inscrire'. At the bottom of the form, there is a small link that says 'Cliquez ici si vous avez déjà un compte'. The footer is dark blue and contains the text 'Mentions légales', 'Top5League © 2026', and 'Contact'.

Inscription

Pseudo :

Joueur préféré :

Email :

Mot de passe :

S'inscrire

[Cliquez ici si vous avez déjà un compte](#)

Mentions légales
Top5League © 2026
Contact

Page connexion

The login screen features a dark blue header with the 'TOP 5 LEAGUE' logo on the left and a hamburger menu icon on the right. The main content area is light yellow and contains a white login form with a blue border. The form has the title 'Connexion' in bold blue text. It includes two input fields labeled 'Email:' and 'Mot de passe :'. Below these fields is a blue button labeled 'Se connecter'. At the bottom of the form, there is a small link that says 'Cliquez ici si vous n'avez pas encore de compte'. The footer is dark blue and contains the text 'Mentions légales', 'Top5League © 2026', and 'Contact'.

Connexion

Email:

Mot de passe :

Se connecter

[Cliquez ici si vous n'avez pas encore de compte](#)

Mentions légales
Top5League © 2026
Contact



Dossier projet – Top 5 League

Desktop :

Page Connexion

[Accueil](#) [Résultats](#) [Les joueurs du mois](#) [Connexion](#) [Inscription](#)

Connexion

Email :

Mot de passe :

[Se connecter](#)

Cliquez ici si vous n'avez pas encore de compte

[Mentions légales](#) [Top5League © 2026](#) [Contact](#)

Page Inscription

[Accueil](#) [Résultats](#) [Les joueurs du mois](#) [Connexion](#) [Inscription](#)

Inscription

Pseudo :

Joueur préféré :

Email :

Mot de passe :

[S'inscrire](#)

Cliquez ici si vous avez déjà un compte

[Mentions légales](#) [Top5League © 2026](#) [Contact](#)

Page d'accueil visiteur

[Accueil](#) [Résultats](#) [Les joueurs du mois](#) [Connexion](#) [Inscription](#)

Comment ça marche ?

1- Découvre les joueurs

Chaque début de mois, les 20 meilleurs joueurs (4 par postes) du mois passé sont sélectionnés en fonction de leur statistique et mis à disposition afin que tu puisses créer ton équipe type.

2- Crées ton 5 majeurs

Tu crées ton 5 majeurs au début du mois en sélectionnant 1 joueur par poste et tu nous mets les statistiques (points, rebonds, passes) que d'après toi chaque joueurs vont faire sur le mois.

3- Remporte des lots

Si tu trouves le 5 tu gagnes le maillot de ton choix ! Pour départager celui qui repartira avec le maillot de son choix si plusieurs personnes l'ont trouvé, on regardera par rapport aux statistiques.

Compose ton 5 du mois

Ca y est c'est le moment ! Lance toi et crée ton 5 majeur du mois, tu as jusqu'à 5 pour le faire et pour nous donner les 3 principales statistiques de tes joueurs afin de devenir le gagnant du TOP5LEAGUE challenge du mois !

[Participe au challenge](#)

Challenge du mois en cours

Top 20 publié : **Oui** Nombre de participants : **32** Statut des votes : **Ouvert**

Le grand gagnant du mois dernier

Il s'agit ce mois-ci de ***** qui est le seul à avoir trouvé le 5 !

Voici le nom des 5 joueurs qui lui ont permis de remporter le maillot de son choix :

John Doe

John Doe

John Doe

John Doe

John Doe

Top 20 du mois



John Doe
Poste : PG PPG : 28.4
APG : 4.2 RPG : 4.7



John Doe
Poste : PG PPG : 28.4
APG : 4.2 RPG : 4.7



John Doe
Poste : PG PPG : 28.4
APG : 4.2 RPG : 4.7



John Doe
Poste : PG PPG : 28.4
APG : 4.2 RPG : 4.7

[Accéder au Top 20 Complet](#)

[Mentions légales](#) [Top5League © 2026](#) [Contact](#)

Page liste des joueurs Visiteur

[Accueil](#)
[Résultats](#)
[Les joueurs du mois](#)
[Connexion](#)
[Inscription](#)

Les joueurs du mois par poste

Les meneurs (PG) :

Les arrières (SG) :

Les ailiers (SF) :

Les ailiers forts (PF) :

Les pivots (C) :

[Mentions légales](#)
[Top5 League © 2020](#)
[Contact](#)

Page résultat visiteur

[Accueil](#)
[Résultats](#)
[Les joueurs du mois](#)
[Connexion](#)
[Inscription](#)

Le grand gagnant du mois dernier

Il s'agit ce mois-ci de ***** qui est le seul à avoir trouvé le 5 et qui repart donc avec le maillot de son choix !

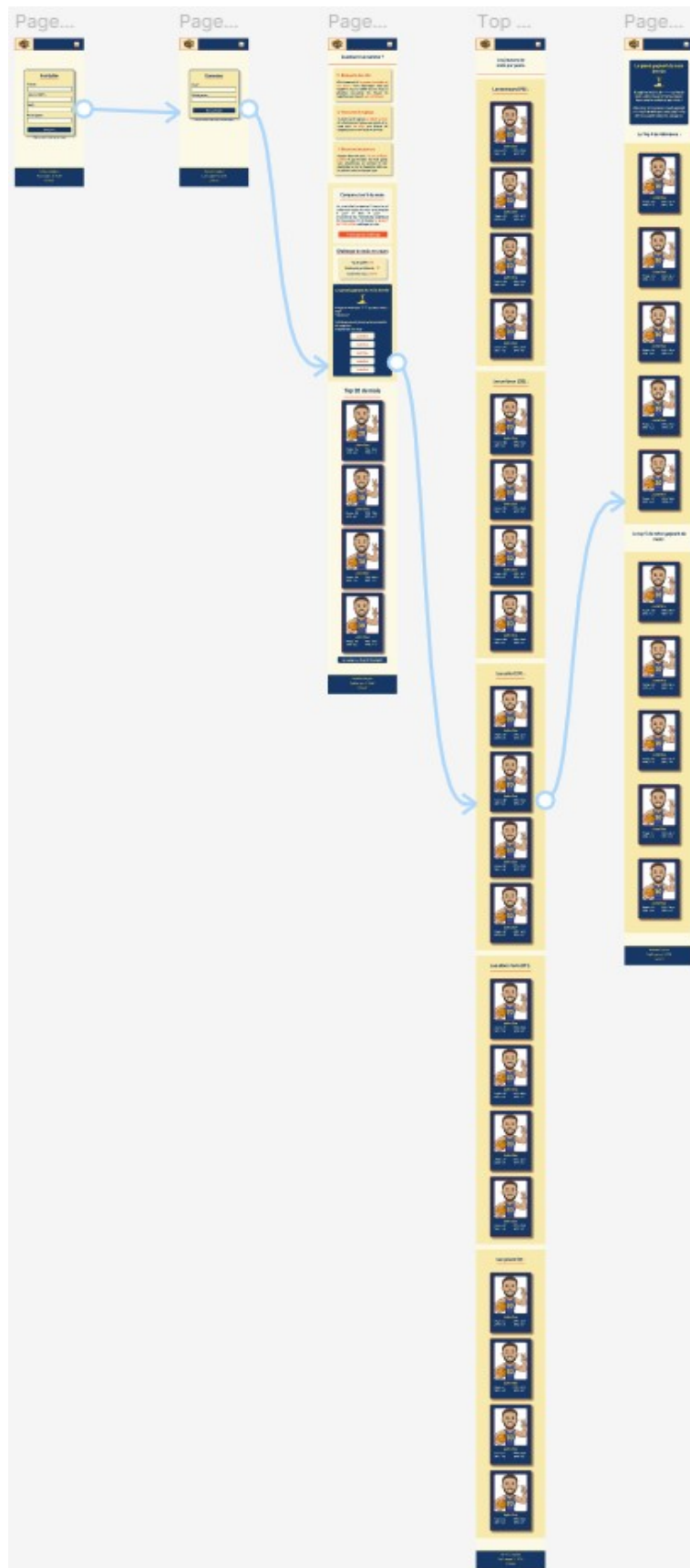
Voici dans ci-dessous le top 5 gagnant et le top 5 de référence mis côte à côte afin de pouvoir mieux les comparer.

Le Top 5 Officiel :

Le top 5 de notre gagnant du mois :

[Mentions légales](#)
[Top5 League © 2020](#)
[Contact](#)

Schéma de l'enchaînement des maquettes :





Référencement SEO

Pour ce qui est du référencement naturel (SEO), le projet a été optimisé afin d'améliorer sa visibilité sur les moteurs de recherche et l'expérience utilisateur.

Pour cela, des balises ont été ajoutées dans le fichier index.jsx :

- Une balise « title » dans lequel le titre du projet et le thème ont été mis ;
- Des balises meta « description » et « keywords » dans lesquels une brève description de l'API et des mots clés de recherche ont été mis ;
- La définition de la langue
- Un favicon créé spécialement pour le site
- L'utilisation d'URL claire et explicite avec React Router
- Une compatibilité responsive grâce à la balise « viewport »
- Une bonne utilisation des balises h1

L'application a également été pensée responsive et accessible afin d'améliorer les performances SEO et l'expérience utilisateur sur mobile et desktop.

Organisations des composants

L'architecture du projet est organisée en plusieurs composants React. Cette approche permet de séparer les responsabilités, de rendre le code plus lisible et de faciliter la maintenance du projet.

Les composants sont organisés par fonctionnalité métier afin de rendre les pages plus claires et d'éviter des fichiers trop lourds ou difficiles à maintenir.

Pour illustrer cela on peut par exemple prendre 4 dossiers significatifs :

- Le dossier « homepage » qui contient les composants de la page d'accueil selon les différents rôles utilisateurs ;
- Les dossiers « header » et « footer » qui contiennent les composants selon leur emplacement dans la page associée afin de pouvoir modifier simplement la partie souhaitée et surtout qui seront directement insérés dans le fichier « App.jsx » puisqu'ils seront réutilisés dans toutes les pages ;
- Le dossier « player-cards » qui contient les cards prédites et les cards officiels qui serviront dans de nombreuses pages ;
- Le dossier « challenge » qui contient toute la logique liée au challenge.

L'application utilise ainsi une architecture basée sur des composants réutilisables, avec une séparation claire entre les pages et les composants. Cette organisation est rendue possible grâce à un système de routage avec React Router permettant de relier les différentes vues de l'application.

Ainsi cette organisation améliore aussi bien la maintenabilité du projet, la réutilisation des composants ainsi que la clarté du code pour le travail en équipe ou les futures évolutions.

L'ensemble du projet suit ainsi une logique de développement moderne adaptée aux applications React.

Capture d'écran d'interface utilisateurs :

Dans cette partie nous verrons d'autres exemple que les maquettes présentées précédemment afin de voir une plus grande variété de contenu du site.

Pour commencer on verra le formulaire de création du top 5 d'un utilisateur puis ensuite le top 5 en question créé présent dans la page « Mon top 5 » (en version mobile et en desktop).

The screenshot shows a web browser window with the URL `localhost:5173/createlineup`. The page title is "Création Top 5". The form is organized into five columns, each representing a basketball position:

- Ailier (SF)**: Nom du joueur (dropdown), Points par match (PPG) (20.0), Passe décisive par match (APG) (5.2), Rebonds par match (RPG) (10.3).
- Meneur (PG)**: Nom du joueur (dropdown), Points par match (PPG) (20.0), Passe décisive par match (APG) (5.2), Rebonds par match (RPG) (10.3).
- Arrière (SG)**: Nom du joueur (dropdown), Points par match (PPG) (20.0), Passe décisive par match (APG) (5.2), Rebonds par match (RPG) (10.3).
- Ailier Fort (PF)**: Nom du joueur (dropdown), Points par match (PPG) (20.0), Passe décisive par match (APG) (5.2), Rebonds par match (RPG) (10.3).
- Pivot (C)**: Nom du joueur (dropdown), Points par match (PPG) (20.0), Passe décisive par match (APG) (5.2), Rebonds par match (RPG) (10.3).

At the bottom center of the form is a button labeled "Créer mon Top 5".

(39) React useState... JSON Web Tokens -... Project Overview | C... (1

Rapport de pixels de l'appareil : Auc

Création Top 5

Ailier (SF)

Nom du joueur :

Points par match (PPG) :

Passe décisive par match (APG) :

Rebonds par match (RPG) :

Meneur (PG)

Nom du joueur :

Points par match (PPG) :

Passe décisive par match (APG) :

Rebonds par match (RPG) :

9) React useState... JSON Web Tokens -... Project Overview | C... (1

Rapport de pixels de l'appareil : Auc

Arrière (SG)

Nom du joueur :

Points par match (PPG) :

Passe décisive par match (APG) :

Rebonds par match (RPG) :

Ailier Fort (PF)

Nom du joueur :

Points par match (PPG) :

Passe décisive par match (APG) :

Rebonds par match (RPG) :

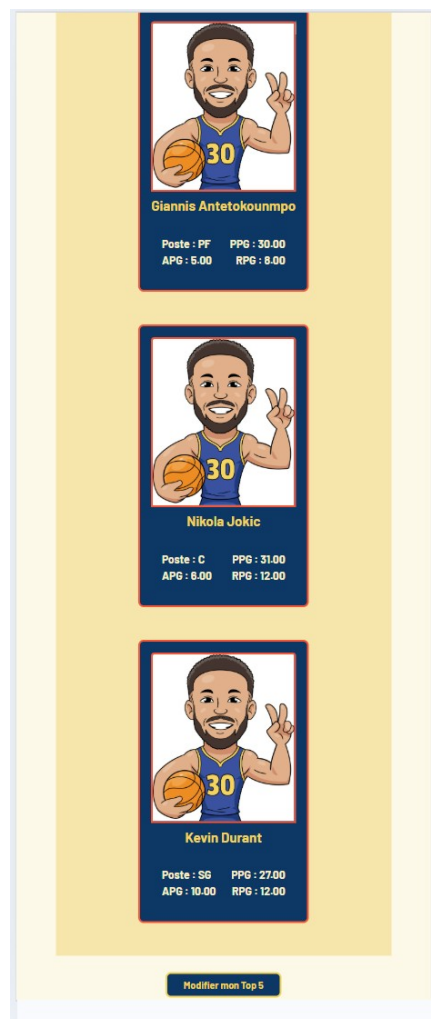
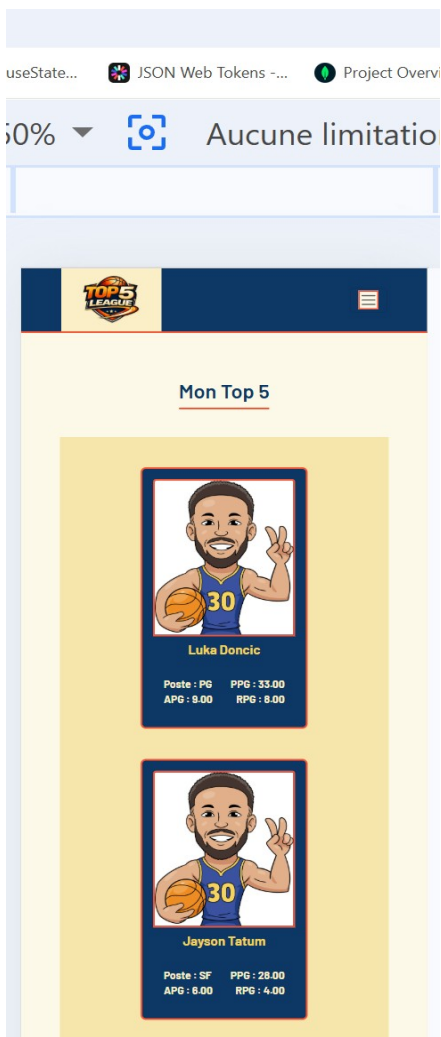
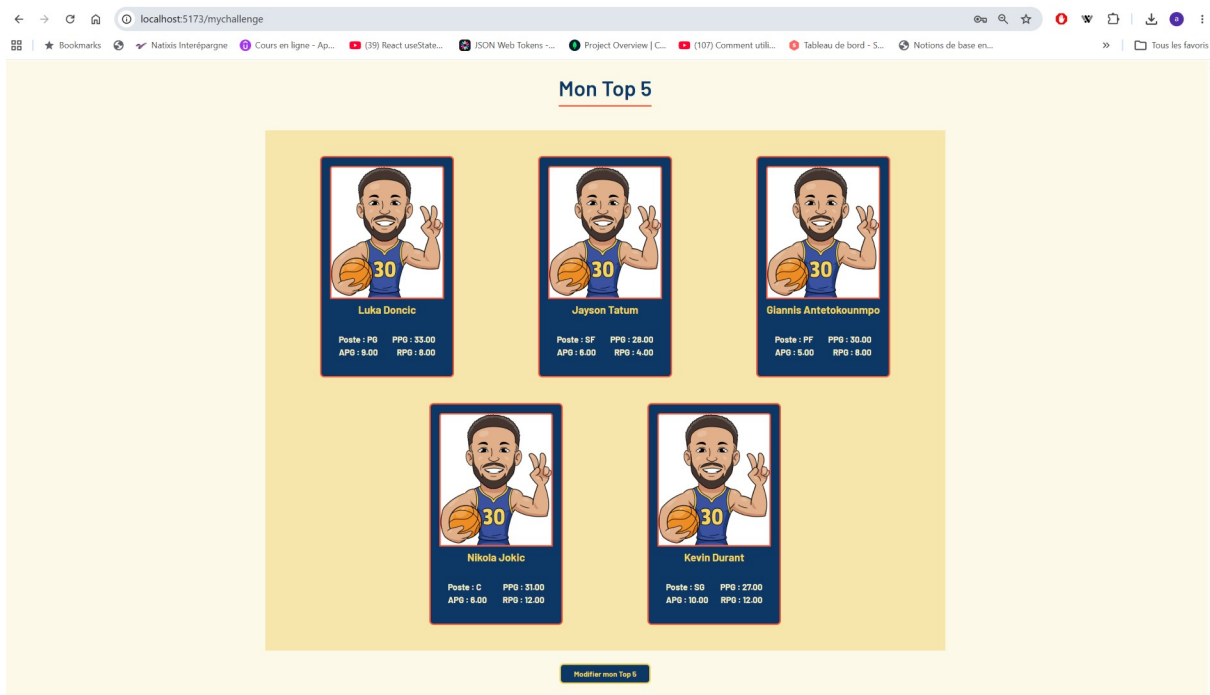
Pivot (C)

Nom du joueur :

Points par match (PPG) :

Passe décisive par match (APG) :

Rebonds par match (RPG) :



Exemple d'interface statique

Les exemples suivants représentent des interfaces statiques de l'application. Ces composants ont principalement un rôle visuel et informatif, avec un contenu fixe qui n'évolue pas ou très peu avec les interactions de l'utilisateur.

Le footer :

```
import { useState } from 'react';
import { Link } from "react-router-dom";
import ContactModal from "../contactModal"

const AppFooter = () => {

  const [showModal, setShowModal]= useState(false);

  return (
    <div className='footer mt-4'>
      <footer className="pt-3 pb-5">
        <div className="container d-flex flex-column flex-md-row justify-content-between align-items-center gap-2 px-5">
          <Link to="/" className="link-footer text-decoration-none ">
            Mentions légales
          </Link>

          <span className=" link-footer">
            Top5League © 2026
          </span>

          <button className=" btn btn-link link-footer" onClick={() => setShowModal(true)} >
            Contact
          </button>
        </div>
      </footer>

      <ContactModal
        show={showModal}
        close={() => setShowModal(false)}
      />
    </div>
  );
};

export default AppFooter;
```


Le composant «Rules» présent dans la page d'accueil utilisateur et visiteur présentant les règles du jeu :

```
const Rules= ()=> {
  return (
    // Section règle du jeu
    <section className="rules-section py-4 mb-5">

      <div className="div-title-rules d-flex justify-content-center mb-5">
        <h1 className="title-part mb-4">Comment ça marche ?</h1>
      </div>

      <div className="rules row g-5">

        <div className="col-12 col-md-4">
          <div className="rules-step h-100 p-3">
            <h5 className="rules-step-title mb-3"> 1- Découvre les joueurs</h5>
            <p className="text-rules">Chaque début de mois, <span className="important">les 20 meilleurs joueurs</span>
            (4 par poste) du mois passé sont sélectionnés en fonction de leurs statistiques et mis à disposition afin que
            tu puisses créer ton équipe type.</p>
          </div>
        </div>

        <div className="col-12 col-md-4">
          <div className="rules-step h-100 p-3">
            <h5 className="rules-step-title mb-3">2- Crée ton 5 majeurs</h5>
            <p className="text-rules">Tu crées ton Top 5 <span className="important">au début du mois</span>
            en sélectionnant 1 joueur par poste et tu indiques
            <span className="important">les statistiques (points, rebonds, passes)</span> que, d'après toi, chaque
            joueur va réaliser en moyenne sur le mois.</p>
          </div>
        </div>

        <div className="col-12 col-md-4">
          <div className="rules-step h-100 p-3">
            <h5 className="rules-step-title mb-3">3- Remporte le maillot de ton choix </h5>
            <p className="text-rules">Si tu trouves le Top 5 <span className="important">
              tu gagnes le maillot de ton choix</span> !
              Pour départager celui qui repartira avec le maillot de son choix en cas d'égalité,
              on regardera celui étant le plus proche des statistiques de
              l'équipe de référence.</p>
          </div>
        </div>
      </div>
    </section>
  )
};

export default Rules;
```

Exemple d'interface dynamique

L'extrait de code suivant correspond à une partie de l'interface de création du Top 5.

Cette partie a été choisie car elle montre le côté dynamique de l'application avec React. Les différents champs du formulaire sont générés automatiquement afin de permettre à l'utilisateur de créer son Top 5 plus facilement.

Lorsque l'utilisateur sélectionne ou modifie un joueur, les données se mettent à jour directement sans recharger la page. Cet extrait montre également la gestion des formulaires et des données utilisateurs avec React.

```
{players.map((player, index) => (
  <div key={player.position} className="col-md-6 col-lg-4">
    <div className="lineup-form p-3 h-100">

      <h4 className="form-title-part text-center mb-1"> {positions[index].label} </h4>
      <br />

      {/* Joueur */}
      <label htmlFor="playerId" className="form-label fw-semibold mt-0">Nom du joueur :</label>
      <select className="form-control" value={player.playerId}
        onChange={(e) =>
          handleChange(index, "playerId", e.target.value)
        } required
      >
        <option value="">Sélectionner un joueur</option>
        {playersName
          .filter(p => p.position === player.position)
          .sort((a,b) => a.fullname.localeCompare(b.fullname, 'fr', { sensitivity: 'base' })))
          .map(
            (p) => (
              <option key={p.id} value={p.id}> {p.fullname} </option>))
          )
      </select>
    </div>
  </div>
)
```

Dans cet extrait, React génère automatiquement les différents champs du formulaire grâce à la fonction map().

Chaque « card » correspond à une position précise du Top 5 (PG, SG, SF, PF et C).

L'utilisateur peut sélectionner un joueur puis renseigner ses statistiques prédites (points, passes décisives et rebonds).

À chaque modification, les données sont mises à jour automatiquement sans rechargement de la page grâce à la gestion des états React.

L'extrait de code ci dessous repose sur la gestion de l'authentification côté client grâce à React. Un système d'authentification (AuthContext) permet de stocker les informations de l'utilisateur connecté et de les utiliser dans toute l'application.

```
// Pour la récupération des données des utilisateurs
export default function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [token, setToken] = useState(localStorage.getItem("token") || null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    async function fetchMe() {
      if (!token) {
        setLoading(false);
        return;
      }
      try {
        const { data } = await api.get("/auth/me");
        setUser(data.data);
      } catch {
        localStorage.removeItem("token");
        setToken(null);
      } finally {
        setLoading(false);
      }
    }
    fetchMe();
  }, [token]);
}
```

Les hooks `useState` permettent de gérer les différents états de l'authentification, notamment les informations de l'utilisateur, le token JWT ainsi que l'état de chargement de l'application.

Le hook `useEffect` exécute automatiquement une vérification lors du chargement de l'application. Si un token est présent dans le stockage local du navigateur (`localStorage`), une requête est envoyée à l'API afin de récupérer les informations de l'utilisateur connecté.

En cas de succès, les données de l'utilisateur sont enregistrées dans l'état global de l'application. En cas d'échec (token expiré ou invalide), le token est supprimé et l'utilisateur est automatiquement déconnecté.

Ce mécanisme permet ainsi d'adapter dynamiquement l'interface selon le statut de connexion de l'utilisateur et de sécuriser l'accès aux différentes fonctionnalités de l'application.

Développement de la partie Back-End :

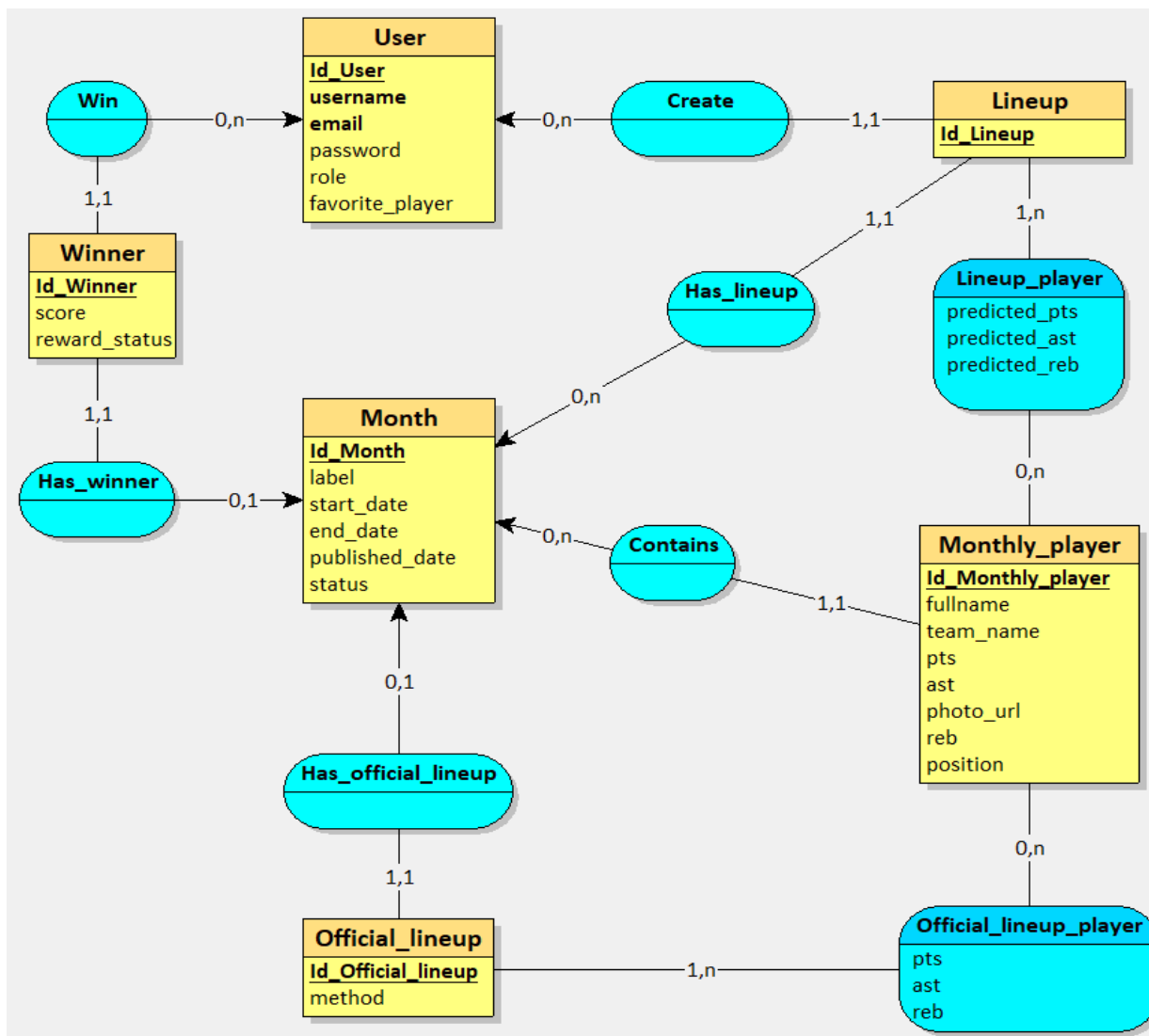
Base de données / MCD / MLD :

Environnement choisis pour la Base de données

Pour la gestion de la base de données du projet, le système de gestion utilisé est MySQL via l'environnement de développement local XAMPP. Cet environnement a été choisis car il permet de disposer rapidement d'un serveur local complet comprenant notamment Apache et MySQL, facilitant ainsi le développement et les tests de l'application.

Le choix d'une base de données relationnelle a été fait car celle ci a permis d'organiser les données du projet de manière structurée et cohérente grâce à des tables reliées entre elles.

MCD

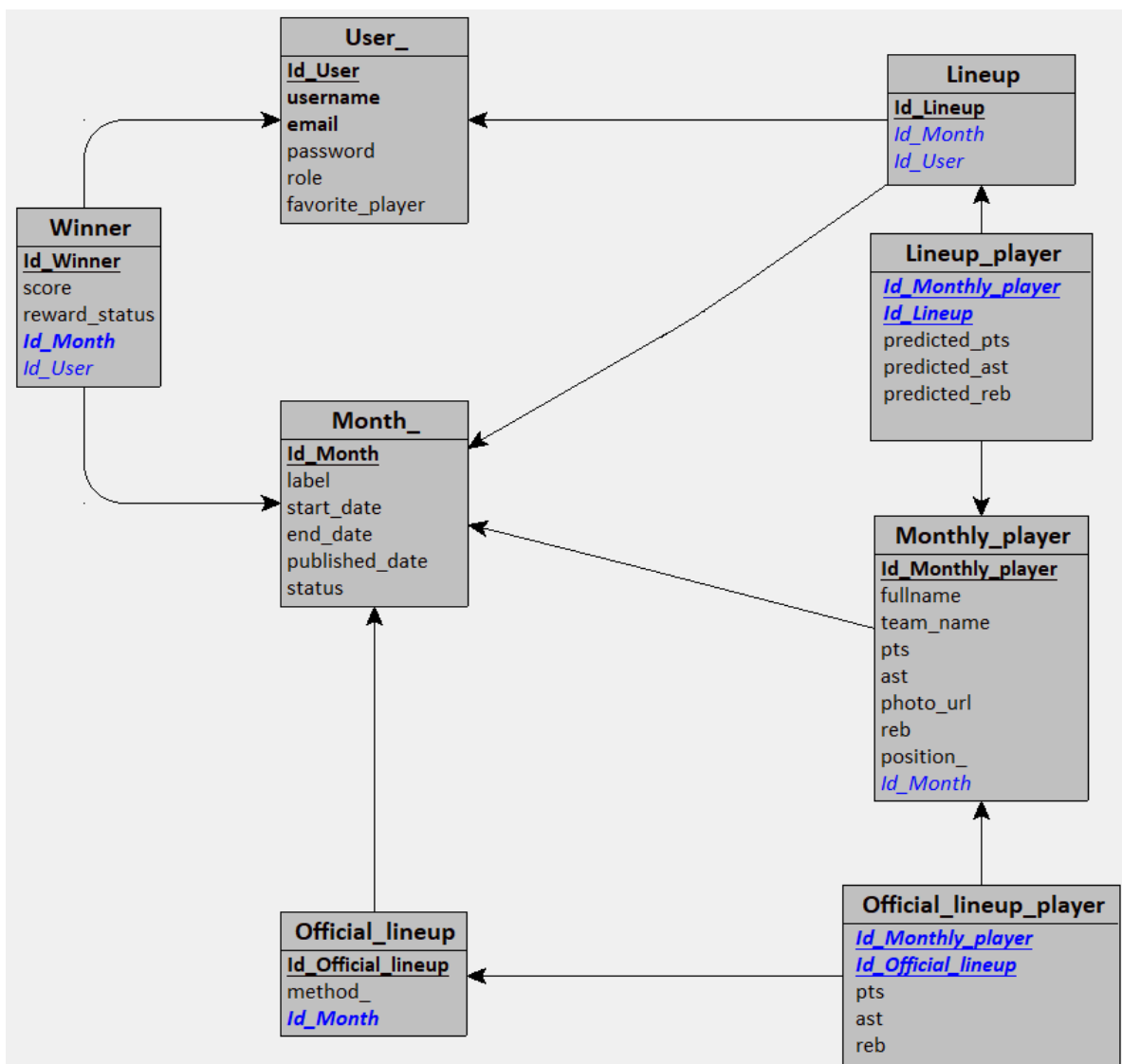


Le MCD présenté ci-dessus permet de représenter les différentes entités utilisées dans l'application ainsi que les relations existantes entre elles.

Il représente notamment les utilisateurs, les mois de challenge, les joueurs du mois ainsi que les différents Top 5 créés dans l'application.

Ce modèle conceptuel a également permis de préparer la création du MLD puis de la base de données relationnelle utilisée dans le projet.

MLD



Le MLD présenté ci-dessus représente l'organisation logique de la base de données du projet.

Il montre les différentes tables utilisées dans l'application ainsi que les clés primaires et étrangères permettant de relier les données entre elles.

Script de création

```

CREATE DATABASE IF NOT EXISTS top5league;
USE top5league;

CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) NOT NULL UNIQUE,
  email VARCHAR(100) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL,
  role ENUM('user','admin') NOT NULL DEFAULT 'user',
  favorite_player VARCHAR(100) NULL
);

CREATE TABLE months (
  id INT AUTO_INCREMENT PRIMARY KEY,
  label VARCHAR(20) NOT NULL UNIQUE,
  start_date DATE NOT NULL,
  end_date DATE NOT NULL,
  publish_date DATE NOT NULL,
  status ENUM('open','closed','published')
  NOT NULL DEFAULT 'open'
);

CREATE TABLE monthly_players (
  id INT AUTO_INCREMENT PRIMARY KEY,
  month_id INT NOT NULL,
  fullname VARCHAR(100) NOT NULL,
  position ENUM('PG','SG','SF','PF','C') NOT NULL,
  team_name VARCHAR(100) NULL,
  pts DECIMAL(5,2) NOT NULL DEFAULT 0.00,
  ast DECIMAL(5,2) NOT NULL DEFAULT 0.00,
  reb DECIMAL(5,2) NOT NULL DEFAULT 0.00,
  photo_url VARCHAR(255) NULL,

  UNIQUE (month_id, fullname),

  FOREIGN KEY (month_id) REFERENCES months(id)
  ON DELETE CASCADE ON UPDATE CASCADE
);

CREATE TABLE lineups (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  month_id INT NOT NULL,

  UNIQUE (user_id, month_id),

  FOREIGN KEY (user_id) REFERENCES users(id)
  ON DELETE CASCADE ON UPDATE CASCADE,

  FOREIGN KEY (month_id) REFERENCES months(id)
  ON DELETE CASCADE ON UPDATE CASCADE
);

CREATE TABLE lineup_players (
  id INT AUTO_INCREMENT PRIMARY KEY,
  lineup_id INT NOT NULL,
  monthly_player_id INT NOT NULL,

  predicted_pts DECIMAL(5,2),
  predicted_ast DECIMAL(5,2),
  predicted_reb DECIMAL(5,2),

  FOREIGN KEY (lineup_id) REFERENCES lineups(id)
  ON DELETE CASCADE ON UPDATE CASCADE,

  FOREIGN KEY (monthly_player_id) REFERENCES monthly_players(id)
  ON DELETE RESTRICT ON UPDATE CASCADE
);

CREATE TABLE official_lineups (
  id INT AUTO_INCREMENT PRIMARY KEY,
  month_id INT NOT NULL UNIQUE,
  method ENUM('manual','auto') NOT NULL DEFAULT 'manual',

  FOREIGN KEY (month_id) REFERENCES months(id)
  ON DELETE CASCADE ON UPDATE CASCADE
);

CREATE TABLE official_lineup_players (
  id INT AUTO_INCREMENT PRIMARY KEY,
  official_lineup_id INT NOT NULL,
  monthly_player_id INT NOT NULL,

  pts DECIMAL(5,2),
  ast DECIMAL(5,2),
  reb DECIMAL(5,2),

  FOREIGN KEY (official_lineup_id) REFERENCES official_lineups(id)
  ON DELETE CASCADE ON UPDATE CASCADE,

  FOREIGN KEY (monthly_player_id) REFERENCES monthly_players(id)
  ON DELETE RESTRICT ON UPDATE CASCADE
);

CREATE TABLE winners (
  id INT AUTO_INCREMENT PRIMARY KEY,
  month_id INT NOT NULL UNIQUE,
  user_id INT NOT NULL,
  score INT NOT NULL DEFAULT 0,
  reward_status ENUM('pending','sent') NOT NULL DEFAULT 'pending',

  FOREIGN KEY (month_id) REFERENCES months(id)
  ON DELETE CASCADE ON UPDATE CASCADE,

  FOREIGN KEY (user_id) REFERENCES users(id)
  ON DELETE CASCADE ON UPDATE CASCADE
);

```

Environnement choisis

Pour la partie back-end, « Node.JS » et le framework « Express » ont été utilisés pour le développement.

Node.JS a été choisi puisqu'il permet d'utiliser le langage JavaScript aussi bien côté front-end que back-end, ce qui rend le développement plus simple et cohérent.

Pour ce qui est d'Express, le framework a été choisi pour sa simplicité, sa rapidité de mise en place mais aussi pour sa compatibilité avec les API REST.

Plusieurs technologies complémentaires ont également été utilisées :

- Express Router pour la gestion des routes ;
- JWT pour l'authentification ;
- bcrypt pour le chiffrement des mots de passe ;
- cors pour sécuriser les échanges entre frontend et backend ;
- dotenv pour la gestion des variables d'environnement.
- express-validator pour la validation et la sécurisation des données envoyées par les utilisateurs ;
- helmet pour renforcer la sécurité de l'API grâce à différents headers HTTP ;
- mysql2 pour permettre la connexion et les échanges avec la base de données MySQL ;
- sequelize comme ORM afin de faciliter les interactions avec la base de données à travers des modèles JavaScript ;

Des outils de développement ont également été utilisés :

- nodemon pour redémarrer automatiquement le serveur lors des modifications ;
- jest et supertest pour les tests unitaires et les tests des routes API ;

Éléments de sécurité

Plusieurs éléments de sécurité ont été mis en place dans la partie back-end afin de protéger l'application, les données utilisateurs ainsi que les échanges entre le front et l'API.

Tout d'abord l'authentification avec JWT qui permet de sécuriser les accès, de protéger certaines routes et de contrôler l'identité des utilisateurs connectés.

Le chiffrement des mots de passe avec bcrypt qui permet de renforcer la sécurité des données utilisateurs en cas d'accès non autorisé à la base de données.

La validation des données avec express validator qui permet de sécuriser les données envoyées par les utilisateurs (champs obligatoire, format de données)

La protection des headers HTTP avec Helmet.

La gestion des variables d'environnement qui permet de cacher les données sensibles en ne les exposant pas directement dans le code source.

Enfin grâce à la séparation des responsabilités et l'architecture du back-end en MVC avec des middlewares afin de rendre l'application plus maintenable et de limiter les risques liés à une mauvaise organisation du code.

L'ensemble de ces mécanismes permet d'obtenir une API back-end plus sécurisée, fiable et adaptée à une application web moderne.

Extrait de code de composants métier

L'extrait de code suivant provient du composant métier `computeWinner()`, qui sert à calculer automatiquement le gagnant du mois.

Cette partie a été choisie car elle représente la logique principale de l'application. Le code compare les joueurs choisis par les utilisateurs avec le Top 5 officiel ainsi que les statistiques réelles des joueurs afin d'attribuer un score à chaque participant.

Une fois les scores calculés, les résultats sont triés automatiquement afin de déterminer le gagnant du challenge.

```
// Calcul du gagnant du mois
const scores = lineups.map((lineup) => {
  let score = 0;
  let statDelta = 0;

  // Comparaison des joueurs Officiel et prédis
  lineup.LineupPlayers.forEach((lp) => {
    if (officialPlayerIds.includes(lp.monthly_player_id)) {
      score += 1;
    }

    // Comparaison des statistiques prédites et Officiel
    const targetStats = statsMap.get(lp.monthly_player_id);
    if (targetStats) {
      statDelta +=
        Math.abs(lp.pts - targetStats.pts) +
        Math.abs(lp.reb - targetStats.reb) +
        Math.abs(lp.ast - targetStats.ast);
    }
  });
  return { lineup, score, statDelta };
});

// Trie des résultat par le score et les statistiques
scores.sort((a,b) => {
  if (b.score !== a.score) {
    return b.score - a.score;
  }
  return a.statDelta - b.statDelta
});
```

Extrait de code de composant d'accès aux données :

L'extrait de code suivant correspond à un composant d'accès aux données de l'application.

Cette partie a été choisie car elle montre la récupération d'un Top 5 utilisateur ainsi que les joueurs associés grâce aux relations mises en place avec Sequelize. Elle permet également de récupérer les statistiques prédites enregistrées dans la table de liaison.

Cet extrait met en avant l'utilisation des relations many-to-many ainsi que la récupération de données liées depuis la base de données.

```
// Récupérer son lineup du mois
static async getMyLineup(user_id) {
  try {
    const month = await monthService.getCurrentMonth();
    const month_id = month.id;

    const lineup = await Lineup.findOne({
      where: { month_id, user_id },
      include: [{ model: MonthlyPlayer, through: { attributes: ["predicted_pts", "predicted_ast", "predicted_reb"] } }],
    });

    if (!lineup) {
      return null;
    }

    return lineup;
  } catch (err) {
    throw new Error(
      `Erreur lors de la récupération du Top 5 ${err.message}`,
    );
  }
}
```

Cette récupération de données est réalisée grâce à la méthode include de Sequelize, qui permet de charger automatiquement les données liées à un modèle.

L'utilisation de through permet également de récupérer les informations présentes dans la table de liaison LineupPlayer, notamment les statistiques prédites par l'utilisateur (predicted_pts, predicted_ast et predicted_reb).

Présentation du jeu d'essai de la fonctionnalité la plus représentative

Pour cette partie, la fonctionnalité de création du Top 5 par un utilisateur va être testée.

Pour ce faire, un nouvel utilisateur nommé « test » a été créé. Son Top 5 a ensuite été créé directement depuis le site afin de vérifier que celui-ci est correctement enregistré à la fois sur l'application et dans la base de données.

L'utilisateur créé possède l'identifiant 9 et le challenge du mois actuellement ouvert possède l'identifiant 2.

Dans un premier temps, on peut constater dans la base de données qu'aucun Top 5 n'existe encore pour cet utilisateur et ce mois.

✓ Affichage des lignes 0 - 4 (total de 5, traitement en 0,0002 seconde(s).)

```
SELECT * FROM `lineups`
```

☐ Profilage [[Éditer en ligne](#)] [[Éditer](#)] [[Expliquer SQL](#)] [[Créer le code source PHP](#)] [[Actualiser](#)]

☐ Tout afficher | Nombre de lignes : 25 ▼ | Filtrer les lignes:

Options supplémentaires

				id	user_id	month_id
☐	Éditer	Copier	Supprimer	1	2	1
☐	Éditer	Copier	Supprimer	4	2	2
☐	Éditer	Copier	Supprimer	2	4	1
☐	Éditer	Copier	Supprimer	5	4	2
☐	Éditer	Copier	Supprimer	3	5	1

Ensuite, le formulaire de création du Top 5 est rempli avec les différentes prédictions des joueurs avant validation afin de pouvoir comparer les données par la suite.

The screenshot shows a web browser window with the URL 'localhost / 127.0.0.1 / top5leagu'. The page title is 'Top5League - Fantasy Basketball'. The browser's address bar shows 'localhost / 127.0.0.1 / top5leagu'. The page content is a form for creating a Top 5 League, divided into five sections: Ailier (SF), Meneur (PG), Arrière (SG), Ailier Fort (PF), and Pivot (C). Each section contains input fields for player name, points per game (PPG), assists per game (APG), and rebounds per game (RPG). The 'Créer mon Top 5' button is at the bottom.

Ailier (SF)	Meneur (PG)	Arrière (SG)	Ailier Fort (PF)	Pivot (C)
Nom du joueur : Jayson Tatum	Nom du joueur : Stephen Curry	Nom du joueur : Kevin Durant	Nom du joueur : Giannis Antetokounmpo	Nom du joueur : Sélectionner un joueur
Points par match (PPG) : 25.1	Points par match (PPG) : 30.2	Points par match (PPG) : 32	Points par match (PPG) : 28.7	Points par match (PPG) : 33
Passe décisive par match (APG) : 5.4	Passe décisive par match (APG) : 6.8	Passe décisive par match (APG) : 3	Passe décisive par match (APG) : 2.4	Passe décisive par match (APG) : 11
Rebonds par match (RPG) : 6.3	Rebonds par match (RPG) : 5.2	Rebonds par match (RPG) : 6	Rebonds par match (RPG) : 11.4	Rebonds par match (RPG) : 17

Créer mon Top 5

Une fois le formulaire envoyé, on constate que le Top 5 a bien été créé grâce au message de confirmation affiché après avoir cliqué sur « Créer mon Top 5 ». Le Top 5 apparaît également correctement sur la page « Mon Top 5 » avec les joueurs et statistiques sélectionnés.



Mon Top 5



Jayson Tatum

Poste : SF PPG : 25.10
APG : 5.40 RPG : 6.30



Stephen Curry

Poste : PG PPG : 30.20
APG : 6.80 RPG : 5.20



Kevin Durant

Poste : SG PPG : 32.00
APG : 3.00 RPG : 6.00



Giannis Antetokounmpo

Poste : PF PPG : 28.70
APG : 2.40 RPG : 11.40



Victor Wembanyama

Poste : C PPG : 33.00
APG : 11.00 RPG : 17.00

Modifier mon Top 5

Enfin, les captures d'écran de la base de données montrent que les données ont correctement été enregistrées dans les tables lineups et lineup_players.

✓ Affichage des lignes 0 - 5 (total de 6, traitement en 0,0002 seconde(s).)

`SELECT * FROM `lineups``

☐ Profilage [Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

☐ Tout afficher | Nombre de lignes : 25 | Filtrer les lignes: Chercher dans cette table | Trier par clé : Aucun(e)

Options supplémentaires

			id	user_id	month_id
☐	Éditer	Copier	Supprimer	1	2
☐	Éditer	Copier	Supprimer	4	2
☐	Éditer	Copier	Supprimer	2	4
☐	Éditer	Copier	Supprimer	5	4
☐	Éditer	Copier	Supprimer	3	5
☐	Éditer	Copier	Supprimer	6	9

✓ Affichage des lignes 25 - 29 (total de 30, traitement en 0,0002 seconde(s).)

`SELECT * FROM `lineup_players``

☐ Profilage [Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

<< < 2 | ☐ Tout afficher | Nombre de lignes : 25 | Filtrer les lignes: Chercher dans cette table | Trier par clé : Aucun(e)

Options supplémentaires

				id	lineup_id	monthly_player_id	predicted_pts	predicted_ast	predicted_reb
☐	Éditer	Copier	Supprimer	26	6	23	25.10	5.40	6.30
☐	Éditer	Copier	Supprimer	27	6	22	30.20	6.80	5.20
☐	Éditer	Copier	Supprimer	28	6	27	32.00	3.00	6.00
☐	Éditer	Copier	Supprimer	29	6	24	28.70	2.40	11.40
☐	Éditer	Copier	Supprimer	30	6	34	33.00	11.00	17.00

Ce jeu d'essai a donc permis de vérifier que la fonctionnalité de création du Top 5 fonctionne correctement, aussi bien au niveau de l'interface utilisateur que de l'enregistrement dans la base de données

Déploiement de l'application

Le projet a été déployé et testé dans un environnement local durant son développement.

Le backend était exécuté avec Node.js et Express tandis que le frontend React était lancé avec Vite. La base de données MySQL était hébergée localement afin de faciliter les phases de développement et de test.

Une mise en ligne complète du projet pourrait cependant être réalisée afin de rendre l'application accessible publiquement. Le frontend pourrait par exemple être déployé sur une plateforme comme Vercel, tandis que le backend Node.js pourrait être hébergé sur Render ou Railway.

La base de données MySQL pourrait également être hébergée sur un service distant afin de permettre la communication entre les différentes parties de l'application en production.

Tests supplémentaire réalisés (tests unitaires)

Des test unitaire ont été réalisés sur chacune des routes afin de vérifier le bon fonctionnement des toutes les fonctionnalités de l'API.

Ces tests ont été réalisé grâce a Jest et SuperTest. SuperTest a été utilisé pour simuler des requêtes HTTP vers l'API Express tandis que Jest permet d'exécuter et d'automatiser les différents scénarios de test.

Ils ont permis notamment de vérifier le bon fonctionnement des routes, les codes de statut, des fonctionnalités métier ou encore la gestion des erreurs.

Veille sur les vulnérabilités de sécurité

Durant le développement du projet, une veille a été réalisée concernant les principales vulnérabilités pouvant affecter l'application.

La première a été portée à la sécurisation de l'authentification des utilisateurs. Afin d'éviter le stockage de mots de passe en clair dans la base de données, les mots de passe ont été hashés grâce à bcrypt. L'authentification a également été sécurisée avec l'utilisation de tokens JWT permettant de protéger les routes privées de l'application.

La deuxième a concerné les injections SQL et les entrées utilisateurs malveillantes. Pour limiter ces risques, les requêtes vers la base de données ont été réalisées via Sequelize, permettant l'utilisation de requêtes préparées, et les données utilisateurs ont été validées avec express-validator.

Le middleware Helmet a également été utilisé afin de renforcer la sécurité des headers HTTP de l'application et limiter certaines vulnérabilités connues du web comme certaines attaques XSS ou l'exposition d'informations sensibles du serveur.

Conclusion

Difficultés rencontrées

Plusieurs difficultés ont été rencontrées au cours du développement du projet.

Côté back-end la première difficulté à être apparue est la mise en place des relations entre les models notamment les relations many to many qui ont été une nouveauté ici.

Le deuxième difficulté a été lors de la création du middleware d'authentification et de la mise en place du token JWT (ici donc concernant la sécurisation de l'application).

La troisième difficulté a été une difficulté de logique métier et a été sur le fait de trouver le gagnant dans le service « Winner ».

Côté front-end, la principale difficulté rencontrée a porté sur les formulaires de modification (avec notamment l'utilisation de « index » et sur la mise en place d'une application responsive (de nombreuses reprises ont dû être faites afin d'arriver au résultat souhaité).

Améliorations possibles

Plusieurs améliorations pourraient être apportées à l'application.

La principale serait plus d'automatisation. Que ce soit pour le changement de mois ou pour l'ajout des 20 nouveaux joueurs du mois, une automatisation via par exemple une application externe pour les 20 joueurs du mois serait une très belle avancée.

Une autre amélioration possible serait pour les utilisateurs, l'accès à un historique sur plusieurs mois avec un classement, leurs top 5, ceux des 3 meilleurs des mois concernés et pourquoi pas leur évolutions de classement par rapport au mois précédent.

Un système de forum pourrait aussi être mis en place dans le but d'augmenter l'engagement des utilisateurs et également attirer tout ceux aimant débattre de sport.

Enfin, la mise en ligne du projet constituerait une étape essentielle afin de rendre l'application accessible aux utilisateurs.

Remerciement

Je tiens à exprimer ma gratitude envers l'ensemble des personnes qui m'ont accompagné durant ma formation de Développeur Web et Web Mobile.

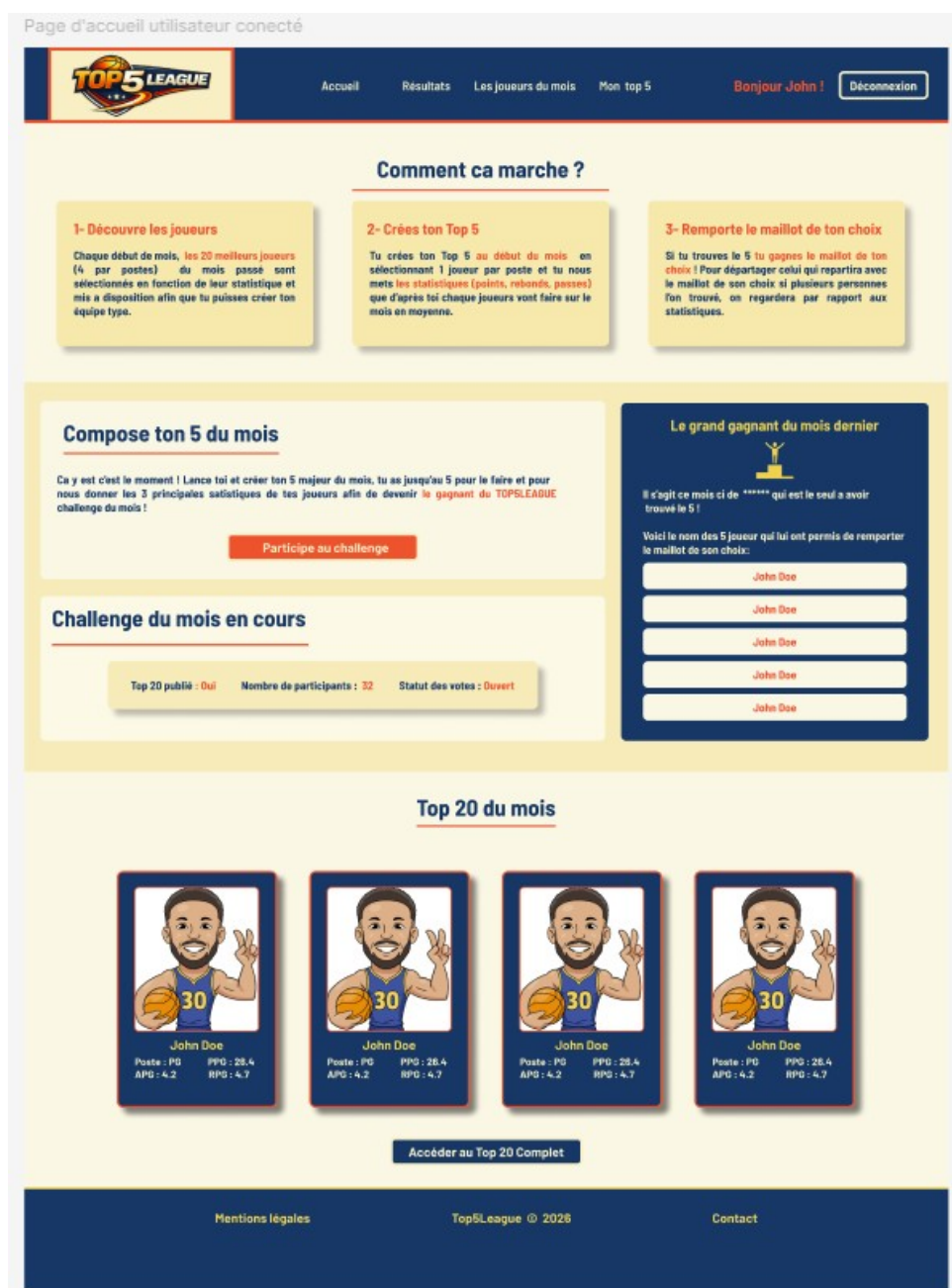
Je remercie tout particulièrement les formateurs pour leur disponibilité, leurs conseils techniques et leur accompagnement tout au long de ce projet. Je souhaite également remercier les intervenants impliqués dans le suivi et l'évaluation de mon parcours, dont les retours ont été précieux pour ma progression.

Enfin, je remercie le centre de formation pour la qualité de l'environnement pédagogique proposé, qui m'a permis de mener à bien ce projet dans des conditions optimales.

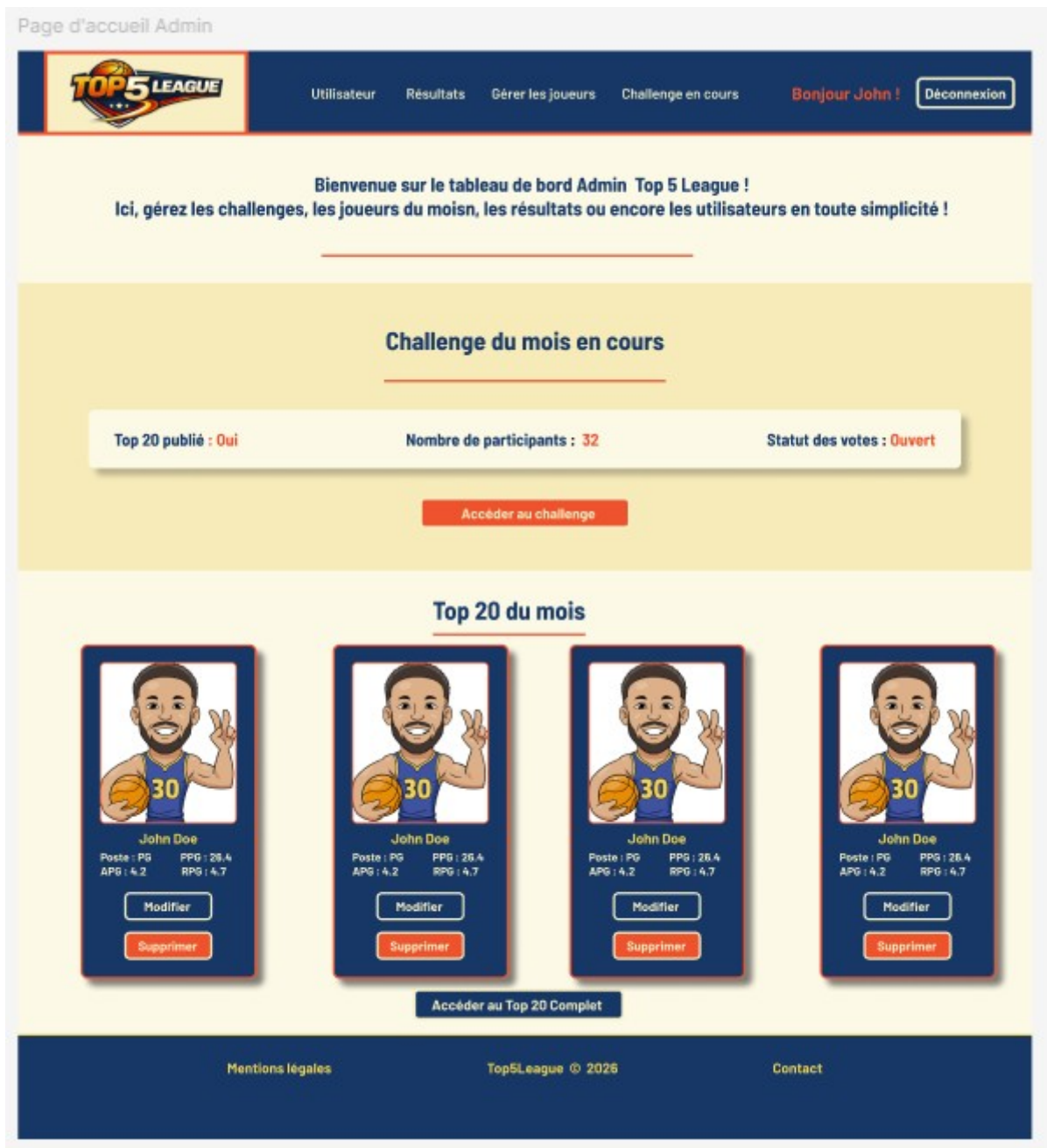
Annexes :**Maquettes des interfaces utilisateur**

Le projet contenant de nombreuses interfaces, il serait trop lourd d'afficher toutes les maquettes. Voici donc 2 maquettes une de la page d'accueil utilisateur et une de la page d'accueil administrateur.

Les autres maquettes, aussi bien desktop que mobile, restent accessibles via les liens Figma présent sous les captures.

Accueil Utilisateur :

Accueil administrateur :



Lien vers maquettes desktop :

<https://www.figma.com/proto/qkqffgoWpHmKT8VvNhddpw/Top-5-League?node-id=2-2&t=CbAyFx3RcyLB2voc-1>

Lien vers maquettes mobile :

<https://www.figma.com/proto/qkqffgoWpHmKT8VvNhddpw/Top-5-League?node-id=0-1&t=CbAyFx3RcyLB2voc-1>

Captures d'écran d'une interface utilisateur et le code correspondant

Pour cette partie, afin de diversifié par rapport à ce que nous avons déjà vu, nous allons voir l'interface de la création d'un joueur officiel et le code correspondant.

```
import { useContext, useEffect, useState } from 'react'
import { useNavigate, useParams } from 'react-router-dom';
import api from '../services/api';
import { AuthContext } from '../context/AuthProviders';

const CreatePlayer= () => {

  const [player, setPlayer] = useState({});
  const {monthId} = useParams();

  const { user, loading } = useContext(AuthContext);
  const navigate = useNavigate();

  // // Protection accès admin
  useEffect(() => {
    if (!loading && user?.role !== "admin") navigate("/");
  }, [user, loading, navigate]);

  // Envoi du nouveau joueur
  const handleSubmit = async (e) => {
    e.preventDefault();

    try {
      await api.post(`/monthlyPlayers/${monthId}`, player);
      alert("Joueur Créer avec succès !");
      navigate("/monthlyplayers");
    } catch (err) {
      console.log(err);
    }
  };

  return (
    <main className="container py-5">
      <div className="title">
        <h1 className="page-title">Création joueur</h1>
      </div>
    </main>
  );
}
```

```

    /* Card */
    <div className="player-card mx-auto p-4">
      <form onSubmit={handleSubmit}>
        /* Poste */
        <div className="poste mb-3">
          <label htmlFor="position" className="form-label fw-semibold">Poste :</label>
          <select className="form-select top5-select" id="position" name="position" value={player.position || ""} onChange={(e) => setPlayer({ ...player, position: e.target.value })} required>
            <option value="PG">Point Guard (PG)</option>
            <option value="SG">Shooting Guard (SG)</option>
            <option value="SF">Small Forward (SF)</option>
            <option value="PF">Power Forward (PF)</option>
            <option value="C">Center (C)</option>
          </select>
        </div>

        /* Nom */
        <div className="nom-du-joueur mb-3">
          <label htmlFor="fullname" className="form-label fw-semibold">Nom du joueur :</label>
          <input type="text" id="fullname" className="form-control top5-input" value={player.fullname || ""} onChange={(e) => setPlayer({ ...player, fullname: e.target.value })} required/>
        </div>

        /* PPG */
        <div className="ppg mb-3">
          <label htmlFor="ppg" className="form-label fw-semibold">Points par match (PPG) :</label>
          <input type="number" step="0.1" id="ppg" className="form-control top5-input" value={player.pts || ""} onChange={(e) => setPlayer({ ...player, pts: e.target.value })} required/>
        </div>

        /* APG */
        <div className="apg mb-3">
          <label htmlFor="apg" className="form-label fw-semibold">Passes décisive par match (APG) :</label>
          <input type="number" step="0.1" id="apg" className="form-control top5-input" value={player.ast || ""} onChange={(e) => setPlayer({ ...player, ast: e.target.value })} required/>
        </div>

        /* RPG */
        <div className="rpg mb-3">
          <label htmlFor="rpg" className="form-label fw-semibold">Rebonds par match (RPG) :</label>
          <input type="number" step="0.1" id="rpg" className="form-control top5-input" value={player.reb || ""} onChange={(e) => setPlayer({ ...player, reb: e.target.value })} required/>
        </div>

        /* PHOTO */
        <div className="photo mb-4">
          <label htmlFor="photo" className="form-label fw-semibold">Photo du joueur:</label>
          <input type="text" id="photo" className="form-control top5-input" value={player.photo_url || ""} onChange={(e) => setPlayer({ ...player, photo_url: e.target.value })}/>
        </div>

        /* Bouton */
        <button type="submit" className="btn btn-blue w-100">Créer le joueur</button>
      </form>
    </div>
  </main>
)
}

export default CreatePlayer;

```

The screenshot shows a web browser window with the URL `localhost:5173/createplayer/2`. The page has a light yellow background and a title **Création joueur** in bold black text. Below the title is a form with a blue border and a light yellow background. The form contains the following elements:

- Poste :** A dropdown menu with 'Point Guard (PG)' selected.
- Nom du joueur :** A text input field.
- Points par match (PPG) :** A number input field.
- Passes décisive par match (APG) :** A number input field.
- Rebonds par match (RPG) :** A number input field.
- Photo du joueur :** A text input field.
- Créer le joueur :** A blue button at the bottom of the form.

Code de composants métier les plus significatifs

Pour ce qui est de cette partie, l'extrait de code de composant métier à déjà été vu dans une partie précédente. Nous verrons donc ici 2 autres composants métier.

```
const jwt = require("jsonwebtoken");

function authenticate(req, res, next){
  const authHeader = req.headers.authorization;
  if(!authHeader || !authHeader.startsWith("Bearer")) {
    return res.status(401).json({message: "Token manquant"});
  }

  const token = authHeader.split(" ")[1];
  // Vérifié le token
  try {
    const payload = jwt.verify(token, process.env.JWT_SECRET);
    req.user = {id: payload.id, role: payload.role}
    next();
  } catch (error) {
    return res.status(401).json({message: "Token invalidé"});
  }
}

function requireAdmin(req, res, next){
  if (req.user.role !== "admin") {
    return res.status(403).json({message: "Accès refusé, administrateur uniquement"});
  }
  next();
}

module.exports = { authenticate, requireAdmin };
```

Middleware d'authentification JWT sécurisant l'accès aux routes de l'application.

Le code présenté ci-dessus correspond au middleware d'authentification utilisé dans le back-end de l'application.

Ce composant permet de sécuriser les routes en vérifiant la présence d'un token JWT dans les requêtes HTTP. Le token est récupéré dans les en-têtes de la requête, puis vérifié à l'aide de la fonction `jwt.verify`.

Si le token est valide, les informations de l'utilisateur sont décodées et ajoutées à la requête (`req.user`), ce qui permet d'identifier l'utilisateur connecté.

En cas d'absence ou d'invalidité du token, l'accès est refusé avec une réponse HTTP 401.

```
// Vérification 1 joueur par poste
const positionMap = new Map();
monthlyPlayers.forEach((player) => {
  positionMap.set(player.id, player.position);
});

positions.forEach((pos) => {
  const hasPos = lineupData.some(
    (p) => positionMap.get(p.playerId) === pos);
  if (!hasPos) {
    throw new Error(`Le joueur pour la position ${pos} est manquant`);
  }
});
```

L'extrait de code suivant correspond à une partie du composant métier de création du Top 5 utilisateur.

Cette partie a été choisie car elle permet de vérifier que le Top 5 respecte bien les règles du challenge avant son enregistrement dans la base de données. Le code contrôle qu'un joueur est bien sélectionné pour chaque poste demandé (PG, SG, SF, PF et C).

Cette validation permet d'éviter la création de compositions invalides et garantit la cohérence des données enregistrées dans l'application.

Code de composants d'accès aux données les plus significatifs

L'extrait de code suivant correspond à un composant d'accès aux données utilisé lors de la création du Top 5 utilisateur.

Cette partie a été choisie car elle permet de récupérer les joueurs sélectionnés par l'utilisateur directement depuis la base de données. Elle vérifie également que les joueurs appartiennent bien au mois actuellement ouvert dans l'application.

Cet extrait montre l'utilisation des requêtes Sequelize ainsi que la récupération de données selon plusieurs conditions.

```
//Récupérer les joueurs du mois en cours
static async getCurrentMonthlyPlayers() {
  try {
    const month = await monthService.getCurrentMonth();

    if (!month) {
      return null;
    }

    const monthlyPlayers = await MonthlyPlayer.findAll({
      where: { month_id: month.id },
      order: [["position", "ASC"]],
    });

    return monthlyPlayers;
  } catch (err) {
    throw new Error(`Erreur lors de la récupération des joueurs : ${err.message}`);
  }
}
```